LMS

SOFTWARE REQUIREMENTS SPECIFICATION

Library Management System

A Web-Based Platform for Automated Library Operations

Document Version	**1.0**
Prepared For	**College / University Project Evaluation**
Document Type	**SRS (IEEE 830 Format)**
Status	**Final Draft**
Date	**August 15, 2026**

CONFIDENTIAL — PREPARED FOR ACADEMIC EVALUATION PURPOSES

Document Control

Revision History

Version	Date	Description	Author
0.1	Draft	Initial requirement gathering and scope definition	Project Team
1.0	Final	Complete SRS prepared for faculty submission and development	Project Team

Approval

Project Title	Library Management System
Project Type	College / University Software Engineering Project
Prepared By	Student Development Team
Reviewed By	Project Guide / Faculty Mentor
Approved By	Head of Department (Pending Evaluation)

Table of Contents

1. Introduction **3**

1.1 Purpose 3

1.2 Document Conventions 3

1.3 Intended Audience and Reading Suggestions 3

1.4 Project Scope 4

1.5 References 4

2. Overall Description **4**

2.1 Product Perspective 4

2.2 Product Features Summary 5

2.3 User Classes and Characteristics 5

2.4 Operating Environment 6

2.5 Design and Implementation Constraints 6

2.6 Assumptions and Dependencies 6

3. System Architecture Overview **7**

4. Use Case Model **8**

5. Functional Requirements (System Features) **9**

6. External Interface Requirements **15**

7. Data Model / Database Design **16**

8. Non-Functional Requirements **18**

9. Appendix A — Data Dictionary **20**

10. Appendix B — Glossary **21**

11. Appendix C — Future Enhancements **21**

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **Library Management System**, a web-based application intended to automate the day-to-day operations of a college or university library. The purpose of this document is to provide a single, complete, and unambiguous reference that can be used by the student development team, the project guide/faculty evaluator, and any future contributor to understand exactly what the system is required to do.

This document follows the general structure recommended by the **IEEE 830 standard** for Software Requirements Specifications, adapted to a scope appropriate for a college-level software engineering project.

1.2 Document Conventions

- Functional requirements are identified using the prefix **FR-XX** and grouped under numbered modules.
- Non-functional requirements are identified using the prefix **NFR-XX**.
- Each requirement is tagged with a priority: **HIGH** **MEDIUM** **LOW**.
- The word **"shall"** is used to indicate a mandatory requirement; **"should"** indicates a desirable but non-mandatory requirement.
- Actors in the system are shown as **Actor Name** .

1.3 Intended Audience and Reading Suggestions

Audience	Recommended Focus
Faculty / Project Evaluators	Sections 1, 2, 4, 5, 8 — scope, use cases, functional and non-functional requirements
Student Developers (Frontend)	Sections 4, 5, 6 — use cases, features, UI requirements
Student Developers (Backend / Database)	Sections 5, 7 — functional requirements and data model
Testers / QA	Sections 5, 8 — acceptance criteria and quality attributes
Project Documentation / Report Writers	Entire document

1.4 Project Scope

The Library Management System replaces manual, register-based library processes with a centralized web application. Students will be able to search the catalog, check real-time availability, request to issue books, return books, view their borrowing history, and see any pending fines. Librarians will manage the book catalog, manage student/member records, process issue and return transactions, track overdue books, and view basic reports through a dedicated dashboard.

The system is scoped for a **single library branch** with two primary user roles (Student/Member and Librarian/Admin). It is a self-contained academic project and does **not** include integration with external library networks (e.g., inter-library loan systems), payment gateways for fines, RFID/barcode hardware, or mobile native applications. These are listed as potential future enhancements in Appendix C.

> **Scope Note:** The project intentionally keeps the feature set realistic for a college team (typically 2–5 students) to design, build, and demo within a single semester, while still resembling a professional, production-style application.

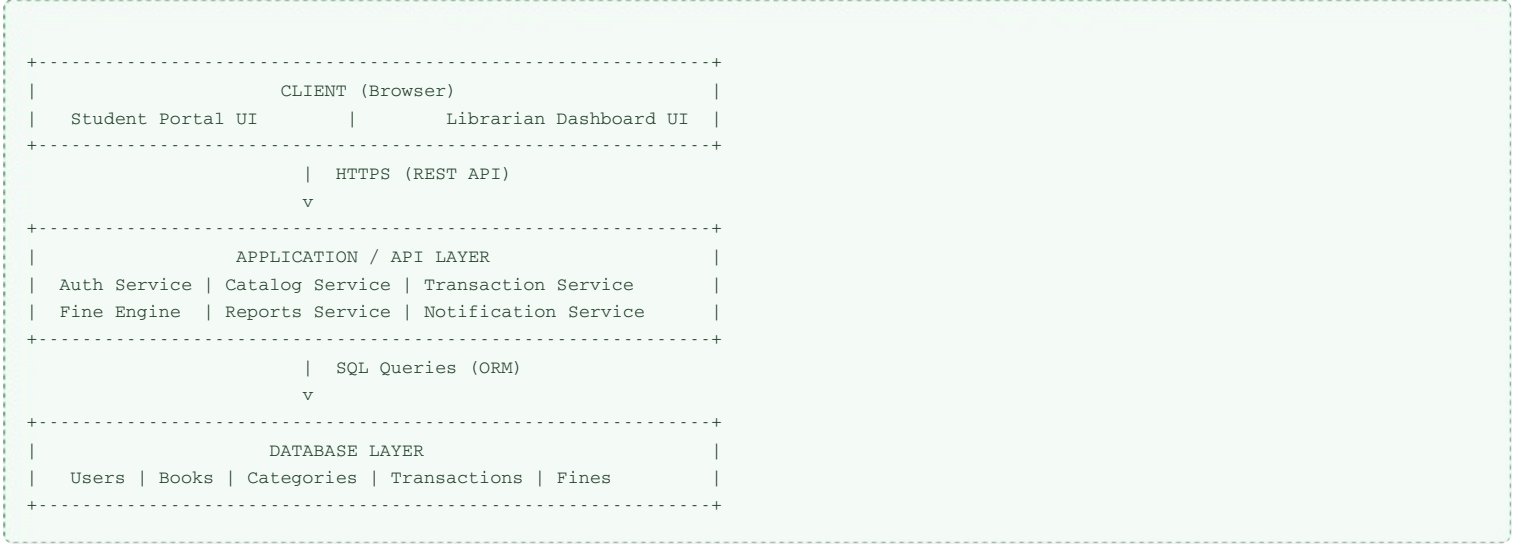
1.5 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- IEEE Std 1016 — Software Design Descriptions (referenced for future design phase)
- Standard relational database design principles (3NF normalization)
- RESTful API design conventions (for future API planning phase)

2. Overall Description

2.1 Product Perspective

The Library Management System is a new, self-contained, three-tier web application. It is not a modification of an existing system. It consists of a browser-based frontend, a backend application/API layer that implements business logic, and a relational database that stores books, users, and transactions.



2.2 Product Features Summary

#	Feature	Primary Actor
1	User Registration, Login & Role-Based Access	Student, Librarian
2	Book Catalog Search & Browsing	Student
3	Book Issue Request & Approval	Student, Librarian
4	Book Return & Fine Calculation	Student, Librarian
5	Borrowing History & Due Date Tracking	Student
6	Book Catalog Management (Add/Edit/Remove)	Librarian
7	Member Management	Librarian
8	Transaction Dashboard & Overdue Tracking	Librarian
9	Basic Reports (issued books, overdue list, fine collection)	Librarian
10	Email/On-screen Due Date Notification	System

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Student / Member	Registered library member who searches for books, requests to issue/return books, and views personal borrowing history and fines.	Basic — general web/app familiarity
Librarian / Admin	Authorized staff member who manages the book catalog, member records, issue/return transactions, and generates reports.	Moderate — comfortable with dashboard-style software
Guest (Unauthenticated)	Any visitor who can view the public book catalog and search availability but cannot issue books without logging in.	Basic

2.4 Operating Environment

- **Client side:** Any modern web browser (Chrome, Firefox, Edge, Safari) on desktop, laptop, or tablet — responsive design.
- **Server side:** Deployable on a standard web/application server (e.g., Node.js/Express, Django/Flask, or Spring Boot — implementation language is a team choice).
- **Database:** Relational database management system (e.g., MySQL / PostgreSQL).
- **Hosting:** Can run on a local development server for demo purposes, or be deployed to a cloud/student hosting platform.

2.5 Design and Implementation Constraints

- The system must be built using technologies familiar to a typical college project team (e.g., HTML/CSS/JS frontend with a relational database backend).
- The project timeline is limited to a single academic semester; therefore, advanced features (payment gateways, RFID, ILS integration) are excluded from the current scope.
- The system shall be developed as a web application; native mobile apps are out of scope.
- All passwords must be stored using a secure hashing mechanism (e.g., bcrypt) — plain-text password storage is not permitted.

2.6 Assumptions and Dependencies

- The library has a single physical branch; multi-branch support is not required.
- Each book title may have multiple physical copies, tracked by a copy/quantity count rather than individual barcodes (barcode/RFID tracking is a future enhancement).
- Users have consistent access to the internet/campus network to access the system.
- The institution will provide (or the team will simulate) an initial dataset of books and members for demonstration purposes.
- Fine amounts and loan durations are configurable by the librarian but default to institution-defined values (e.g., ₹2/day fine, 14-day loan period).

3. System Architecture Overview

The system follows a standard **3-tier architecture**, which separates presentation, business logic, and data storage. This separation keeps the project modular and easy to divide among team members (e.g., one member on frontend, one on backend/API, one on database design).

3.1 Presentation Tier

Two main interfaces: a **Student Portal** (catalog search, issue requests, history) and a **Librarian Dashboard** (catalog management, member management, transactions, reports). Built using standard web frontend technologies with a responsive layout.

3.2 Application / Logic Tier

Exposes a REST API consumed by the frontend. Responsible for authentication, authorization (role-based), business rules (loan period, fine calculation, availability checks), and coordination between the catalog, transaction, and notification logic.

3.3 Data Tier

A relational database stores all persistent data: users, books, categories, transactions, and fines. Details of the schema are provided in Section 7 (Data Model).

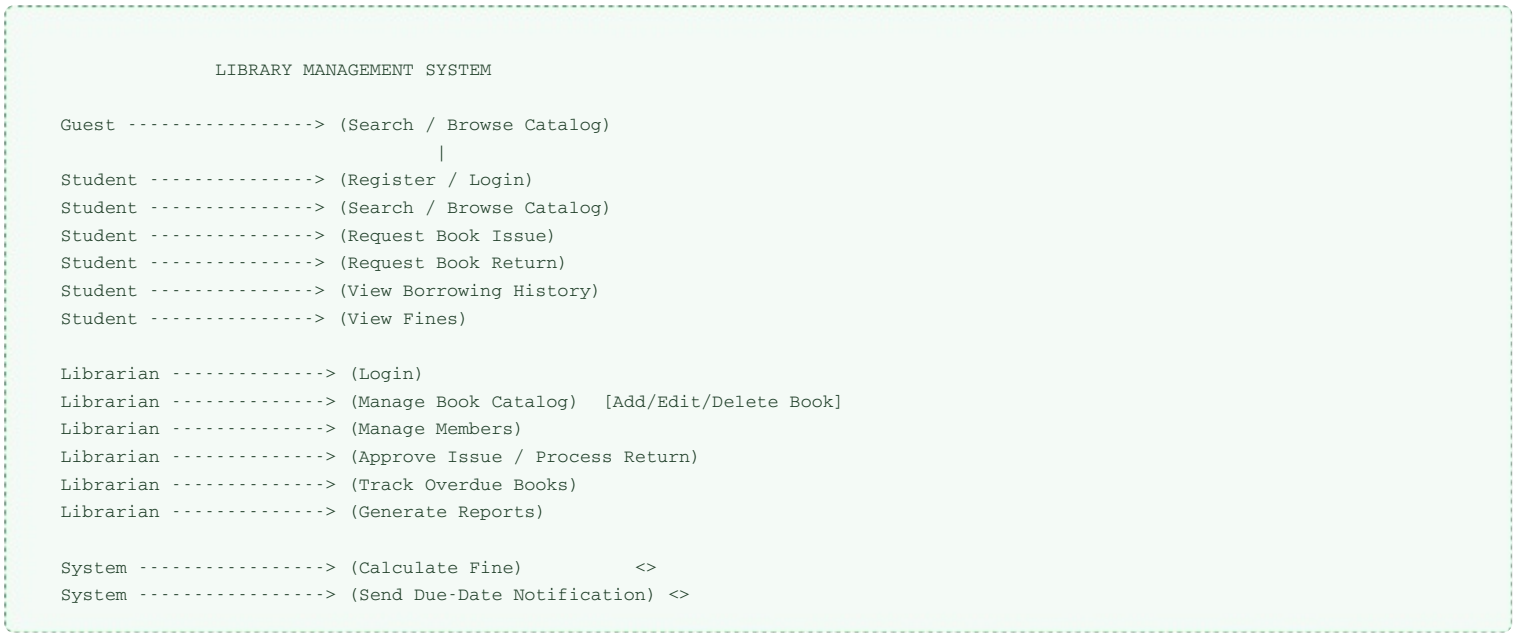
> **Note for Development Team:** The exact technology stack (e.g., React vs. plain HTML/JS, Node.js vs. Django) is a design-phase decision and intentionally left open in this SRS, which focuses on *what* the system must do rather than *how* it is coded.

4. Use Case Model

4.1 Actors

Student / Member | **Librarian / Admin** | **Guest** | **System (automated)**

4.2 Use Case Diagram (Textual Representation)



4.3 Use Case Summary Table

ID	Use Case	Actor(s)	Linked Requirement
UC-01	Register Account	Student	FR-01
UC-02	Login / Logout	Student, Librarian	FR-02
UC-03	Search & Browse Catalog	Guest, Student	FR-05, FR-06
UC-04	Request Book Issue	Student	FR-08
UC-05	Approve / Process Issue	Librarian	FR-09
UC-06	Return Book	Student, Librarian	FR-10
UC-07	Calculate Fine	System	FR-11
UC-08	View Borrowing History	Student	FR-12
UC-09	Manage Book Catalog	Librarian	FR-14 – FR-16
UC-10	Manage Members	Librarian	FR-17, FR-18
UC-11	View Reports	Librarian	FR-19
UC-12	Send Due-Date Notification	System	FR-20

4.4 Sample Use Case Description — UC-04: Request Book Issue

Use Case ID	UC-04
Actor	Student / Member
Precondition	Student is logged in; the requested book has at least one available copy.
Main Flow	1. Student searches for a book in the catalog. 2. Student selects a book and clicks "Issue / Borrow". 3. System verifies availability and that the student has not exceeded the maximum allowed books or has pending unpaid fines. 4. System creates a transaction record with status "Issued" and a calculated due date. 5. System decrements the available copy count. 6. Confirmation is shown to the student.
Alternate Flow	If no copies are available, the system shows "Currently Unavailable" and offers to notify the student when the book is

	returned (future enhancement).
Postcondition	A new transaction is recorded; catalog availability count is updated.

5. Functional Requirements (System Features)

Functional requirements are grouped into six modules: Authentication & Access Control, Catalog & Search, Issue & Return Management, Fine Management, Student Self-Service, and Librarian Administration.

5.1 Module: Authentication & Access Control

FR-01 Student Registration

Description	The system shall allow a new student to register using name, email/roll number, and password.
Inputs	Name, email/roll number, password, confirm password
Outputs	New member account created with role "Student"
Priority	HIGH

FR-02 Login / Logout

Description	The system shall authenticate registered users (Student or Librarian) via email/roll number and password, and shall redirect each role to its respective dashboard.
Priority	HIGH

FR-03 Role-Based Access Control

Description	The system shall restrict Librarian-only features (catalog management, member management, reports) so that they are inaccessible to the Student role.
Priority	HIGH

FR-04 Password Reset

Description	The system should allow users to reset a forgotten password via a registered email link.
Priority	LOW

5.2 Module: Catalog & Search

FR-05 Search Books

Description	The system shall allow users to search the book catalog by title, author, ISBN, or category, with results returned in real time.
Priority	HIGH

FR-06 View Book Details & Availability

Description	The system shall display book details (title, author, publisher, ISBN, category, cover, total copies, available copies) and show real-time availability status ("Available" / "Out of Stock").
Priority	HIGH

FR-07 Filter & Sort Catalog

Description	The system should allow filtering the catalog by category/genre and sorting by title, author, or availability.
Priority	MEDIUM

5.3 Module: Issue & Return Management

FR-08 Request Book Issue

Description	The system shall allow a logged-in student to request to issue an available book, subject to a maximum-books-allowed limit and no outstanding unpaid fines.
Priority	HIGH

FR-09 Librarian Approves / Processes Issue

Description	The system shall allow the librarian to confirm/process an issue request (or issue directly at the counter), automatically setting the issue date and due date (default loan period, e.g., 14 days), and decrementing available copies.
Priority	HIGH

FR-10 **Return Book**

Description	The system shall allow a student to request a return, or a librarian to directly mark a book as returned, updating the transaction status to "Returned", recording the return date, and incrementing available copies.
Priority	HIGH

5.4 Module: Fine Management

FR-11 **Automatic Fine Calculation**

Description	The system shall automatically calculate a fine when a book is returned after its due date, using a configurable per-day rate (default ₹2/day), and shall record the fine against the student's account.
Priority	HIGH

FR-13 **Mark Fine as Paid**

Description	The system shall allow the librarian to mark an outstanding fine as "Paid" once settled, so the student may issue books again.
Priority	MEDIUM

5.5 Module: Student Self-Service

FR-12 **View Borrowing History**

Description	The system shall allow a student to view a complete history of currently issued and previously returned books, including issue date, due date, return date, and any fines.
Priority	HIGH

FR-20 **Due-Date Reminder**

Description	The system should notify a student (on-screen and/or via email) when a borrowed book's due date is approaching (e.g., within 2 days) or has passed.
Priority	LOW

5.6 Module: Librarian Administration

FR-14 **Add New Book**

Description	The system shall allow the librarian to add a new book to the catalog with title, author, ISBN, category, publisher, and number of copies.
Priority	HIGH

FR-15 **Edit / Update Book**

Description	The system shall allow the librarian to edit existing book details, including adding or removing copies.
Priority	HIGH

FR-16 **Remove / Deactivate Book**

Description	The system shall allow the librarian to remove a book from the active catalog (soft delete) if it is not currently issued.
Priority	MEDIUM

FR-17 **Manage Member Records**

Description	The system shall allow the librarian to view, add, edit, and deactivate student/member accounts.
Priority	HIGH

FR-18 **View All Active Transactions**

Description	The system shall provide the librarian a dashboard listing all currently issued books, including borrower name, due date, and overdue status highlighted distinctly.
Priority	HIGH

FR-19 **Generate Basic Reports**

Description	The system shall allow the librarian to generate summary reports: total books issued (by date range), currently overdue books, and total fines collected.
Priority	MEDIUM

6. External Interface Requirements

6.1 User Interfaces

- **Login / Registration Screen:** Simple form with role-aware redirection after login.
- **Student Dashboard:** Search bar, catalog grid/list view, "My Borrowed Books" widget, fine summary.
- **Book Details View:** Cover image placeholder, metadata, availability badge, "Issue" button.
- **Librarian Dashboard:** Navigation panel with Catalog, Members, Transactions, and Reports sections; summary cards (total books, books issued today, overdue count).
- UI shall follow a clean, consistent color theme and be responsive across desktop and tablet screen sizes.

6.2 Hardware Interfaces

No specialized hardware is required. The system runs on standard computers/laptops with a network connection and a modern web browser. Barcode/RFID scanner integration is out of scope for this version (see Appendix C).

6.3 Software Interfaces

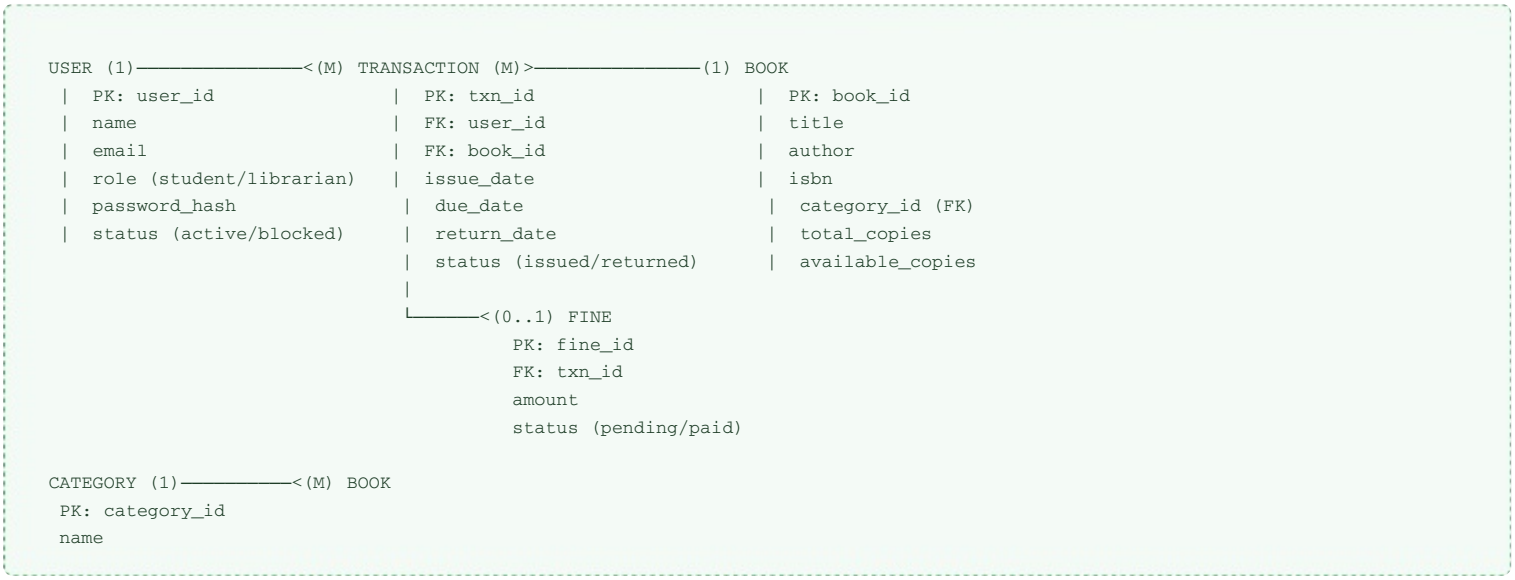
Interface	Purpose
Relational Database (MySQL/PostgreSQL)	Persistent storage of users, books, transactions, and fines
REST API (JSON over HTTPS)	Communication between frontend and backend
SMTP Email Service (optional)	Sending due-date reminders and password reset links

6.4 Communication Interfaces

All client-server communication shall occur over HTTPS. Data shall be exchanged in JSON format between the frontend and the backend API.

7. Data Model / Database Design

7.1 Entity-Relationship Overview (Textual)



7.2 Entity Descriptions

Entity	Description	Key Attributes
User	Represents a student/member or librarian account	user_id (PK), name, email, password_hash, role, status
Book	Represents a book title in the catalog	book_id (PK), title, author, isbn, category_id (FK), total_copies, available_copies
Category	Book genre/category classification	category_id (PK), name
Transaction	Records an issue/return event	txn_id (PK), user_id (FK), book_id (FK), issue_date, due_date, return_date, status
Fine	Records a fine linked to a late-returned transaction	fine_id (PK), txn_id (FK), amount, status

7.3 Relationship Summary

- One **User** can have many **Transactions** (1 : M).
- One **Book** can appear in many **Transactions** over time (1 : M).
- One **Transaction** can result in at most one **Fine** (1 : 0..1).
- One **Category** can classify many **Books** (1 : M).

Note: The schema above is normalized to Third Normal Form (3NF) and is intended as a starting point for the database design phase; exact column types and constraints will be finalized during implementation.

8. Non-Functional Requirements

NFR-01 **Performance**

Requirement	Catalog search results shall be displayed within 2 seconds for a database of up to 5,000 book records under normal load.

NFR-02 **Security**

Requirement	Passwords shall be stored using a secure one-way hash (e.g., bcrypt). All communication shall use HTTPS. Role-based access control shall prevent students from accessing librarian-only functions.

NFR-03 **Usability**

Requirement	A first-time user shall be able to search for a book and understand its availability within 3 clicks, without external instructions.

NFR-04 **Reliability**

Requirement	The system shall correctly maintain book availability counts consistent with issued/returned transactions at all times (no double-issuing of the last available copy).

NFR-05 **Maintainability**

Requirement	The system shall be built using a modular architecture (separate layers for UI, business logic, and data access) to allow future features to be added with minimal changes to existing code.

NFR-06 **Availability**

Requirement	The system shall be available during standard library working hours for demo/evaluation, with graceful error handling for any downtime.

NFR-07 **Portability**

Requirement	The web application shall function correctly on the latest two major versions of Chrome, Firefox, and Edge.

NFR-08 **Scalability**

Requirement	The database schema shall support growth to at least 10,000 books and 2,000 members without requiring redesign.

9. Appendix A — Data Dictionary

Field	Table	Type	Description
user_id	User	Integer (PK)	Unique identifier for a user
role	User	Enum	"student" or "librarian"
status	User	Enum	"active" or "blocked"
book_id	Book	Integer (PK)	Unique identifier for a book title
isbn	Book	String	International Standard Book Number
total_copies	Book	Integer	Total physical copies owned by the library
available_copies	Book	Integer	Copies currently available to issue
txn_id	Transaction	Integer (PK)	Unique identifier for an issue/return event
issue_date	Transaction	Date	Date the book was issued
due_date	Transaction	Date	Date by which the book must be returned
return_date	Transaction	Date (nullable)	Actual date the book was returned
status	Transaction	Enum	"issued", "returned", or "overdue"
amount	Fine	Decimal	Fine amount owed
status	Fine	Enum	"pending" or "paid"

10. Appendix B — Glossary

SRS
Software Requirements Specification — this document.

Member / Student
A registered library user who can borrow books.

Librarian
Staff user responsible for managing the catalog and transactions.

Issue
The act of lending a book copy to a member.

Overdue
A book not returned by its due date.

Fine
Monetary penalty charged for late return of a book.

Catalog
The complete, searchable list of books held by the library.

API
Application Programming Interface — the communication contract between frontend and backend.

11. Appendix C — Future Enhancements (Out of Current Scope)

- Barcode / RFID-based book check-in and check-out
- Online fine payment via payment gateway integration
- Book reservation queue / "notify me when available"
- Multi-branch library support
- Native mobile application (Android/iOS)
- AI-based book recommendations based on borrowing history
- Integration with external Inter-Library Loan (ILL) networks